LAB - 3

System Calls in C

Due Date : 4/30/2026

Lab Overview:

For this lab, we will be learning how to execute some system calls in C. In Linux systems programming, system calls are the main way our program interacts with the OS kernel. System calls are how a program enters the kernel to perform some task. Programs use system calls to perform a variety of operations such as: creating processes, doing network and file IO, and much more. In this lab, we will learn how to use system calls to implement a novel command (**lfcat**) for our pseudo-shell. lfcat() is a novel command that lists all files and their contents.

(Note: The method used to implement the command in this lab is the same as expected for other commands in the project. 1)

Make sure you don't mix the lab source files and directories with the project (files such as command.h, command.c, and output.txt could create confusion). Keep the lab and project work in separate folders.

Core Tasks:

1. Implement the program main with a stub function for lfcat.
2. List all files in the current dir using a loop.
3. Redirect STDOUT to a file with freopen().
4. Write the contents of each file to STDOUT.
5. Test your code with Valgrind for memory leaks.

Goal:

List the contents of all readable files in the current directory to an output file, using **write** and **freopen**. (excluding source file and output file)

Task Details:

1. Implement the program main with a stub function for lfcat**.**

a. For this task, you can edit your main file from project 1. Implement the behavior

shown in Fig. 1. First, the program will display a prompt, followed by collecting user input as discussed in Project 1. The program will also contain the ability to write all stdout to a file named "output.txt". For this step, it is sufficient to use a stub for the lfcat function that prints out a message such as the one shown in fig. 1. **You do not need to do all of the error checking, but it is strongly advised that you do to give you the best footing for Project 1.**

```
dewiy@ix-dev: ~/CS415/Labs/Fall-2023/Lab2/sol 147$ ./lab2
>>> ls
Error: Unrecognized command!
>>> cat
Error: Unrecognized command!
>>> lfcat
>>> exit
dewiy@ix-dev: ~/CS415/Labs/Fall-2023/Lab2/sol 148$
```

Fig. 1: Function stub and calling the function (with error checking)

(You **don't have to** worry about executing multiple commands with ";")

2. In lfcat: As an intermediate step to help you get started, you can first just list all files in the current dir using a loop.
   a. In your implementation, you must utilize the following system calls:
      i. getcwd(2) : http://man7.org/linux/man-pages/man2/getcwd.2.html
      ii. opendir(3): http://man7.org/linux/man-pages/man3/opendir.3.html
      iii. readdir(3): http://man7.org/linux/man-pages/man3/readdir.3.html
      iv. closedir(3): http://man7.org/linux/man-pages/man3/closedir.3.html b.
   See Fig. 2 below for how the listing would look in your code at this step (format for listing is recommended for the project). (You can use print statements at this intermediate step.)

```
dewiy@ix-dev: ~/CS415/Labs/Fall-2023/Lab2/sol/files 202$ ../lab2
>>> lfcat
Listing all the files in the current directory
File: 1_poem.txt
File: 2_lyrics.txt
File: 3_DE_Code.py
>>> exit
dewiy@ix-dev: ~/CS415/Labs/Fall-2023/Lab2/sol/files 203$
```

Fig.2: Listing all entries

3. Redirect STDOUT to output.txt

a. You can use the **freopen(3p)** function below to redirect standard output to a file (all output to standard output will now be saved in the output.txt file).
Hint: use the 'stdout' file stream (in C language)

4. Write the contents of each file to standard output.
   a. Now that we have the filenames, edit your function to write these filenames to standard output using the write system call **write(2)**.
   Hint: use the 'STDOUT_FILENO' file descriptor (in kernel)
   b. In the loop from step 2:
      i. Open the file for reading using **fopen(3)**.
      Hint: the name is specified by ***d->d_name***
      ii. Read in the current file. You must use **getline(3)**
      iii. Write the filename and contents to standard output using **write(2)**.
      Hint: see the attached output.txt for format and sample output
      iv. Close the file using **fclose(3)**. You may also need to null-assign your buffer/file descriptors.
      v. Write 80 "-" characters then repeat the loop.
   c. Clean your code to get rid of any print statements in the lfcat function. See the provided output.txt for the format
   d. Your program can quit right after lfcat() is executed or wait for an "exit" command.
5. Test your code with Valgrind for memory leaks.

Submission Requirements:

Note: This lab should be straightforward if you have already made good progress on your Project 1. The main should work the same as in the project, so feel free to copy-paste your project 1 main.

To receive points for this lab, the student must do the following:

1. Attend the lab and make sure your attendance is recorded.
2. Submit your **output.txt**. (It **must** be the same as the attached output.txt; the order of the files does not matter)
3. Valgrind output with leak-check and mem-check showing no memory leaks.
4. Submit your lab folder to Canvas by the deadline.
5. Lab folder must contain:
   i. main.c
   ii. command.c
   iii. output.txt
   iv. makefile: the command to build your code should be:

1. gcc main.c command.c (make sure to use command.h)

v. screenshot of building, running, and running with Valgrind, and system information (screenfetch)

vi. Other supporting files

The naming convention of the zip/tar file while uploading to Canvas

- Firstname_lastname_image_LabX (an example is given below)
  - firstname: alex
  - lastname: summers
  - image: ex. Debian, Ubuntu Vbox, etc.
  - Submission for: Lab3
  - So the name of the zip/tar file should be:

  alex_summers_DebainUTM_Lab3.zip